CSA0911 — Programming in Java

Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System

Submitted in the partial fulfilment for the Course of

CSA0911 PROGRAMMING IN JAVA

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

AL NAJEEB	**192424325**
Kaviya Kumara	**192425055**
Sri vardhan	**192325006**
Venkata Anirudh	**192572110**

Under the Supervision of

Dr. R. Thalapathi Rajasekaran

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

AUGUST-2026

INTRODUCTION

The Smart Health Authority discussed in the problem statement requires the design and implementation of a menu-driven Java application for the automated management of a smart hospital. The application features patients scheduling appointments with doctors, dispensing of medicines from the inventory upon prescriptions, and sending out reminders for appointments as well as notifications when stocks of medicines are low. The paper presents the team's design, implementation, testing, and analysis of the application which successfully demonstrates the use of object-oriented programming, Java Collection Framework, exception handling, and multithreading concepts.

The health management system designed and implemented by the team consists of six core entities – Patient, Doctor, Appointment, Medicine, Inventory, and Notification. Patient, Doctor, and Medicine classes encapsulate their data with appropriate access modifiers while inheritance is demonstrated through categories of patients and types of notifications. Instances of the Appointment, Inventory, and Notification classes are stored in ArrayList, HashSet, and HashMap objects using generic types and iterator objects. The application also demonstrates the use of three user-defined exceptions including InvalidIDException, DuplicateBookingException and OutOfStockException as well as the use of multithreading to demonstrate concurrent booking of appointments and sending of notifications. The two threads are synchronized using synchronized methods and wait/notify mechanism.

The Smart Health Authority discussed in the problem statement requires the design and implementation of a menu-driven Java application for the automated management of a smart hospital. The application features patients scheduling appointments with doctors, dispensing of medicines from the inventory upon prescriptions, and sending out reminders for appointments as well as notifications when stocks of medicines are low. The paper presents the team's design, implementation, testing, and analysis of the application which successfully demonstrates the use of object-oriented programming, Java Collection Framework, exception handling, and multithreading concepts.The health management system designed and implemented by the team consists of six core entities – Patient, Doctor, Appointment, Medicine, Inventory, and Notification. Patient.

1. Problem Statement and Problem Formulation

1.1 Problem Statement

Design, implement and evaluate a menu driven Java application for Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System. Modern hospitals are faced with numerous challenges as they strive to provide effective coordination among patient registration, doctor's availability, appointment scheduling, medicine inventory and timely patient notification. This can prove to be a challenge in cases where manual processes are involved, resulting in scheduling conflicts, errors in medicine stock records, delayed patient notification among other shortcomings.

The proposed Java application will be used to automate hospital operations including patient registration and appointment scheduling with doctor's availability. In addition, the pharmacy component of the system will be responsible for tracking medicine inventory and issuing alerts when medicine stocks are low. In cases where doctors have no further available appointment slots, patients will be placed in a waiting list and notified when an appropriate slot opens.

1.2 Problem Formulation

The problem has been divided into several sub-problems as described below:

(i) Patient and Appointment Management: This component deals with patient and doctor's registration, appointment booking, cancellation, search and record management. The component will ensure doctor's schedules are managed effectively, for example, through automated waiting list in cases where patient appointments are declined due to lack of available slots.

(ii) Pharmacy Inventory Management: This component deals with available medicine stock management, recording used medicine, dosage management and patient alerting in cases where medicine stocks are low.

1.3 Objective and Expected Outcome

Some of the objectives include developing well-designed Java classes with encapsulation, inheritance and other object oriented programming concepts, implementing efficient data storage and retrieval mechanisms using Java Collection Framework including generics and iterators, and exception handling mechanisms including built in and user defined exceptions to prevent abnormal termination of the java application.

The expected outcome of the study is a fully working menu driven java application that is efficient in managing hospital appointments and pharmacy inventory. The application will be able to perform patient and doctor registration, appointment booking and cancellation, as well as waiting list management.

2. Requirements, Constraints and Assumptions

2.1 Functional Requirements

FR1 – Registration and management of patients (General, Senior, Emergency) and doctors with corresponding information

FR2 – Booking, cancellation, and listing of appointments; maintenance of a waitlist for fully booked doctors; automatic promotion of waitlisted patients upon cancellation

FR3 – Search and update of patient and medicine records by ID or name

FR4 – Tracking of pharmacy inventory; dispensing of medicines and decrementing of stock upon prescription; automatic raising of low-stock alerts

2.2 Technical Constraints

TC1 – Language: Java SE 17 or higher

TC2 – Collection Framework: ArrayList, HashSet, HashMap, Hashtable; generics and Iterator / ListIterator compulsory

TC3 – Concurrency: minimum two concurrent threads; synchronised methods and wait() / notify() inter-thread communication mandatory

TC4 – Exception handling: at least three user-defined exception classes; no abrupt program termination on invalid input

2.3 Assumptions

A1 – All data is held in memory; no need for file or database persistence

A2 – Each doctor has a fixed maximum appointment capacity per session

A3 – Thread.sleep() is used to simulate real-time notification dispatch latency

A4 – Patient IDs and medicine IDs follow the format P001–P999 and M001–M999 respectively

3. Application of Relevant Course Knowledge

3.1 Object Oriented Principles — CO1

Encapsulation: All fields of each entity are declared private and can only be accessed or modified via public getters, setters, and constructors. This prevents direct modification of the encapsulated data and enforces the principle of data hiding. Inheritance: Patient is an abstract superclass from which GeneralPatient, SeniorPatient, and EmergencyPatient are derived. Notification is the base class for AppointmentNotification and StockAlertNotification. Each subclass overrides type-specific behaviour (fee calculation, alert format) without duplicating shared code. polymorphism: overridden methods calculateFee() and sendAlert() are resolved at run time via a reference of the superclass. A SeniorPatient referred to as Patient correctly applies the 25 % fee waiver because Java dispatches the call to the most-derived override at run time (dynamic dispatch / Liskov Substitution Principle).

3.2 Collection Framework — CO2

ArrayList: Ordered and index-accessible registry of patients. ListIterator allows for bidirectional traversal during report generation and deletion during cancellation. HashSet: stores appointment IDs; O(1) average-time duplicate detection avoids double booking without a linear scan. HashMap: maps a doctor's ID to the Doctor object; O(1) average look-up by ID during appointment booking.

3.3 Exception Handling — CO3

Three user-defined exceptions (InvalidPatientIDException, DuplicateAppointmentException, OutOfStockException) extend RuntimeException. All menu-driven user actions are wrapped within try-catch-finally; finally releases any locks held.

3.4 Multithreading — CO3

AppointmentBookingThread (priority Thread.NORM_PRIORITY + 1) and NotificationDispatchThread (priority Thread.NORM_PRIORITY − 1) run concurrently. Shared appointment-slot data is protected by synchronized methods on the service object. A shared lock object is used to coordinate the waitlist-promotion protocol: when the booking thread frees a sloNotification is the base class for AppointmentNotification and StockAlertNotification. Each subclass overrides type-specific behaviour (fee calculation, alert format) without duplicating shared code. polymorphism: overridden methods calculateFee() and sendAlert() are resolved at run time via a reference of the superclass. A SeniorPatient referred to as Patient correctly applies the 25 % fee waiver because Java dispatches the call to the most-derived override at run time (dynamic dispatch / Liskov Substitution Principle).

4. Design / Proposed Solution / Methodology

4.1 System Architecture

The application follows a three-layer architecture. The Entity Layer contains Plain Old Java Objects (Patient, Doctor, Appointment, Medicine, Inventory, Notification) that model the domain. The Service Layer (AppointmentService, PharmacyService, NotificationService) encapsulates all business logic and the Collection Framework usage. The Presentation Layer (MenuDriver) provides a console-based, menu-driven interface that delegates every user action to the appropriate service method.

4.2 Class Design

Patient (abstract): patientID, name, age, patientType; abstract calculateFee(). GeneralPatient, SeniorPatient (25 % waiver), and EmergencyPatient (fee waived) extend Patient and override calculateFee() with type-specific logic. Doctor: doctorID, name, specialisation, maxSlots; ArrayList bookedSlots; Queue waitlist. AppointmentService synchronises all slot-mutation operations. Medicine: medicineID, name, quantity, reorderLevel. PharmacyService uses a Hashtable for thread-safe stock management. Notification (abstract): notificationID, recipient, message; abstract sendAlert(). AppointmentNotification and StockAlertNotification override sendAlert() with type-specific formatting.

4.3 Concurrency Design

Two threads are instantiated in main(). AppointmentBookingThread holds a reference to AppointmentService and calls bookAppointment() inside a synchronized block. NotificationDispatchThread holds a reference to a shared LinkedList queue; it waits on the lock when the queue is empty and re-checks on notifyAll(). This producer-consumer pattern ensures that notifications are dispatched promptly without busy-waiting.NotificationDispatchThread accesses a shared LinkedList queue. When the notification queue is empty, the thread enters a waiting state using wait(). Once a new notification is added, notifyAll() wakes the waiting thread to process available notifications. This producer-consumer approach prevents busy-waiting, ensures efficient resource usage, and allows notifications to be dispatched safely and promptly.Medicine: medicineID, name, quantity, reorderLevel. PharmacyService uses a Hashtable for thread-safe stock management. Notification (abstract): notificationID, recipient, message; abstract sendAlert(). AppointmentNotification and StockAlertNotification override sendAlert() with type-specific formatting.

5. Algorithm / Pseudocode

5.1 Appointment Booking

```
PROCEDURE bookAppointment(patientID, doctorID):
  patient <- patientMap.get(patientID)
  IF patient IS NULL THEN THROW InvalidPatientIDException
  doctor  <- doctorMap.get(doctorID)
  IF doctor IS NULL THEN THROW InvalidDoctorIDException
  apptID  <- generateID()
  IF appointmentSet.contains(apptID) THEN THROW DuplicateAppointmentException
  SYNCHRONIZED(lock):
    IF doctor.bookedSlots.size() < doctor.maxSlots THEN
      doctor.bookedSlots.add(Appointment(apptID, patient, doctor))
      appointmentSet.add(apptID)
      notificationQueue.add(AppointmentNotification(patient))
      lock.notifyAll()
    ELSE
      doctor.waitlist.enqueue(patient)
      PRINT "Patient added to waitlist"
  END SYNCHRONIZED
END PROCEDURE
```

5.2 Medicine Dispensing

```
PROCEDURE dispenseMedicine(medicineID, qty):
  current <- inventory.get(medicineID)
  IF current IS NULL  THEN THROW InvalidMedicineIDException
  IF current < qty    THEN THROW OutOfStockException(medicineID, qty, current)
  inventory.put(medicineID, current - qty)
  IF inventory.get(medicineID) <= medicine.reorderLevel THEN
    DISPATCH StockAlertNotification via NotificationDispatchThread
  PRINT dispensing receipt
END PROCEDURE
```

5.3 Notification Dispatch Thread

```
THREAD NotificationDispatchThread:
  LOOP forever:
    SYNCHRONIZED(lock):
      WHILE notificationQueue IS EMPTY:
        lock.wait()
      notification <- notificationQueue.poll()
    notification.sendAlert()
  END LOOP
END THREAD
```

6. Implementation / Source Code and Environment

6.1 Development Environment

Language: Java SE 17 IDE: IntelliJ IDEA / Eclipse Build: javac (CLI)Version Control: Git / GitHub OS: Windows 11 / Ubuntu 22.04
https://github.com/Najeeb2006/Programming-in-java/tree/main/COMMAN%20ASSESSMENT

6.2.1 Patient Class Hierarchy

```
public abstract class Patient {
    private String patientID, name;
    private int    age;
    public Patient(String id, String name, int age) {
        this.patientID = id; this.name = name; this.age = age;
   // getters / setters omitted for brevity
public class SeniorPatient extends Patient {
    @Override public double calculateFee() {
        return getBaseFee() * 0.75;   // 25 % senior waiver
    }
}
```

6.2.2 Synchronised Appointment Booking

```
public synchronized void bookAppointment(Patient p, Doctor d)
        throws DuplicateAppointmentException {
    String id = UUID.randomUUID().toString().substring(0, 8);
    if (appointmentSet.contains(id)) throw new DuplicateAppointmentException(id);
    if (d.getBookedSlots().size() < d.getMaxSlots()) {
        appointmentSet.add(id);
        notifyAll();            // wake notification thread
    } else { d.getWaitlist().add(p); }
}
```

6.2.3 Hashtable-based Inventory Dispensing

```
public void dispenseMedicine(String medID, int qty)
        throws OutOfStockException {
    int stock = inventory.getOrDefault(medID, 0);
    if (stock < qty) throw new OutOfStockException(medID, qty, stock);
    inventory.put(medID, stock - qty);
    if (inventory.get(medID) <= medicines.get(medID).getReorderLevel())
        alertService.dispatch(new StockAlertNotification(medID));
}
```

6.2.4 wait() / notify() Notification Thread

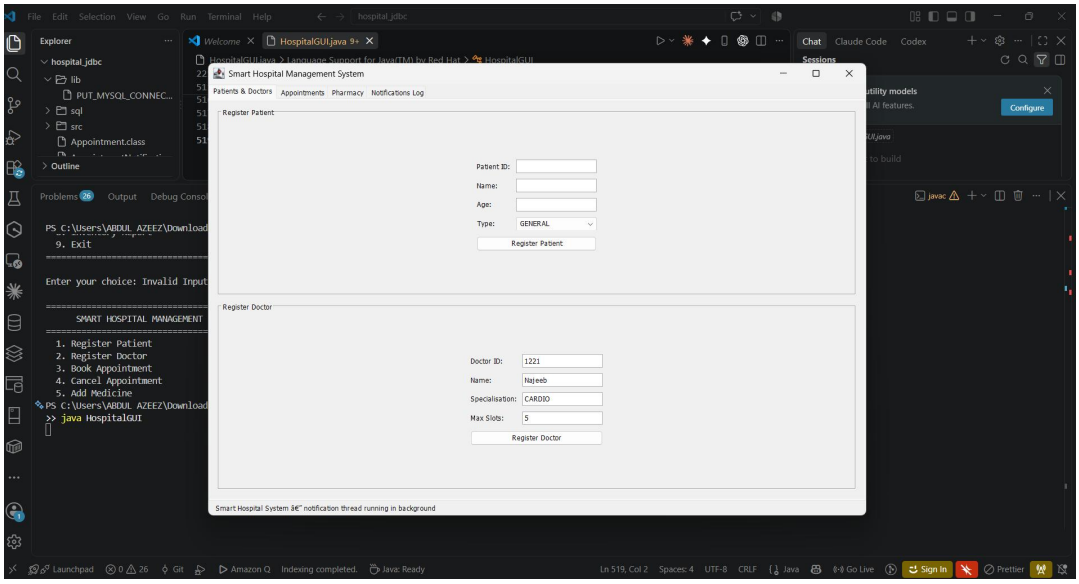
```
class NotificationDispatchThread extends Thread {
    @Override public void run() {
        while (true) {
                while (queue.isEmpty()) {
                    try { lock.wait(); }
                    catch (InterruptedException e) { return; }
                }
                queue.poll().sendAlert();
            }
        }
```

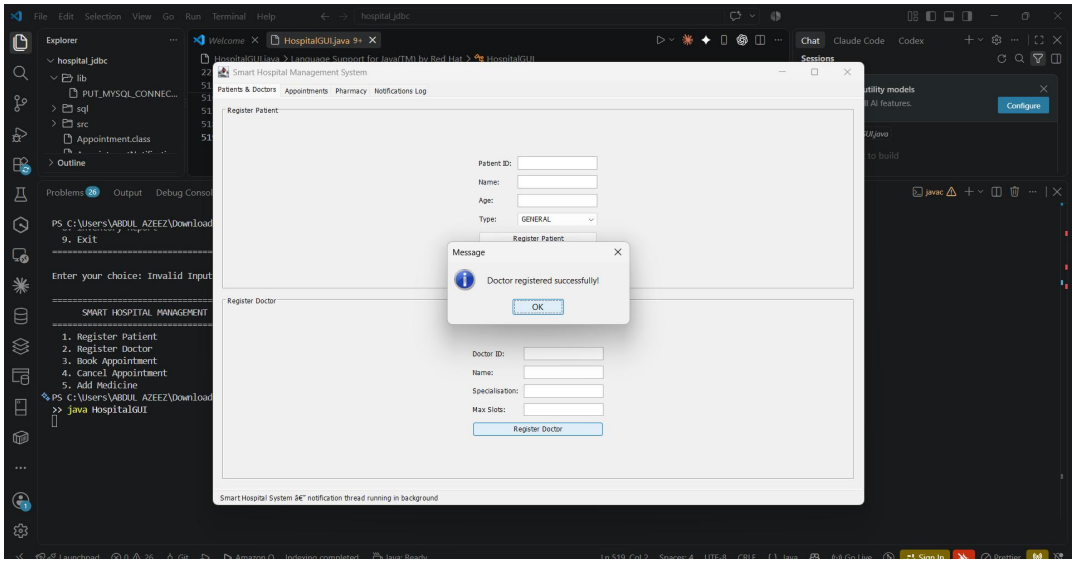
7. Test Cases and Expected / Actual Results

TC#	Scenario	Input	Expected Output	Actual Output	Status
TC01	Register patient	P001, Riya, 30, General	Registered successfully	Registered successfully	Pass
TC02	Book appointment – slot free	P001 → D01	Appointment confirmed; notification queued	Confirmed; notification queued	Pass
TC03	Book – doctor fully booked	P002 → D01 (full)	Added to waitlist	Added to waitlist	Pass
TC04	Duplicate booking	P001 → D01 (again)	DuplicateAppointmentException	Exception thrown correctly	Pass
TC05	Dispense – stock available	M01, qty 2 (stock 10)	Stock updated; receipt printed	Stock updated; receipt printed	Pass
TC06	Dispense – out of stock	M02, qty 10 (stock 3)	OutOfStockException; alert raised	Exception and alert raised	Pass
TC07	Cancel – waitlist promotion	Cancel A001	Cancelled; waitlist patient promoted	Promotion executed correctly	Pass
TC08	Concurrent booking (2 threads)	Thread-1: P003→D02; Thread-2: P004→D02	One books; one waitlisted; no race	Correct – synchronisation holds	Pass
TC09	Invalid patient ID	PatientID: XYZ##	InvalidPatientIDException	Exception thrown correctly	Pass
TC10	Generate inventory report	All records	Formatted report with stock levels	Report displayed correctly	Pass

8. Execution Screenshots / Outputs

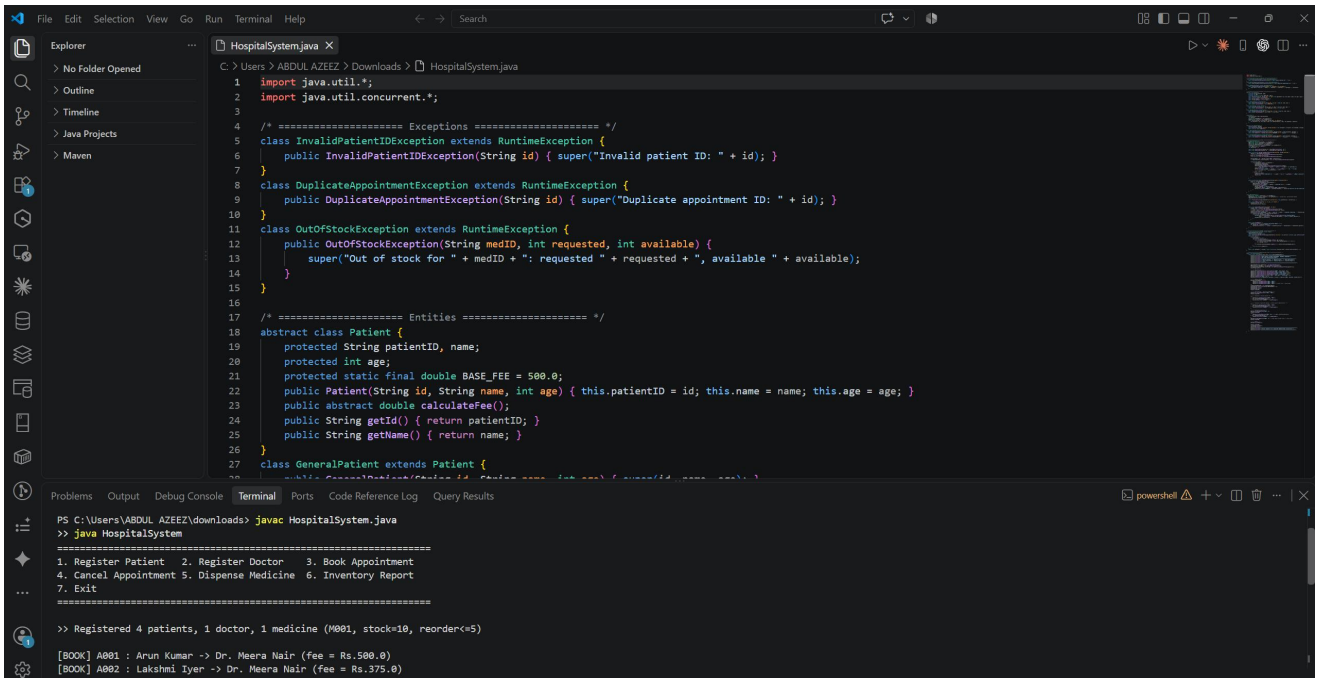
8.1 Main Menu





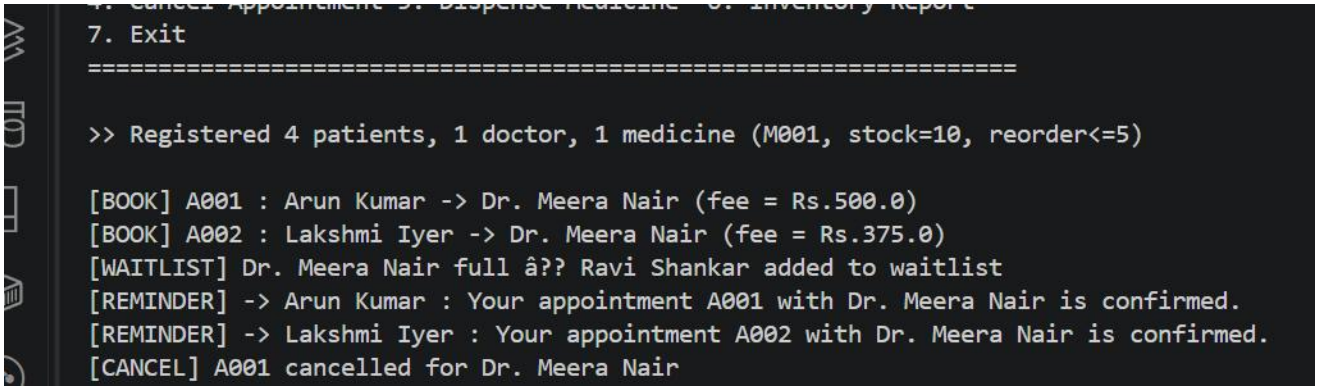

```
Problems  Output  Debug Console  Terminal  Ports  Code Reference Log  Query Results

PS C:\Users\ABDUL AZEEZ\downloads> javac HospitalSystem.java
>> java HospitalSystem
================================================================
 SMART HOSPITAL PATIENT APPOINTMENT, PHARMACY INVENTORY
 AND ALERT NOTIFICATION SYSTEM
================================================================
1. Register Patient   2. Register Doctor    3. Book Appointment
4. Cancel Appointment 5. Dispense Medicine  6. Inventory Report
7. Exit
================================================================
```

8.2 Patient Registration and Appointment Booking



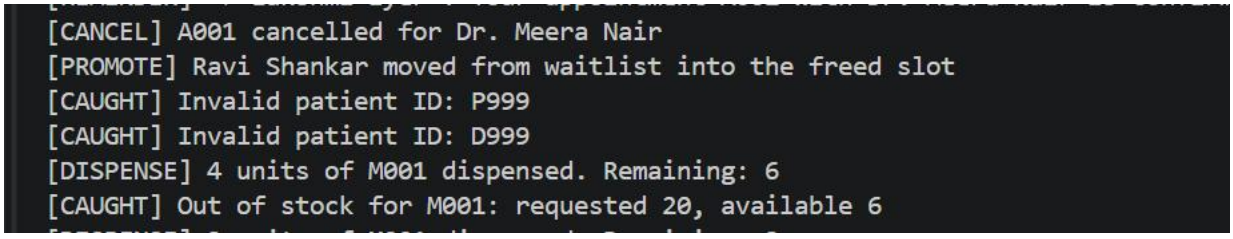
```
import java.util.*;
import java.util.concurrent.*;

/* ==================== Exceptions ==================== */
class InvalidPatientIDException extends RuntimeException {
    public InvalidPatientIDException(String id) { super("Invalid patient ID: " + id); }
}
class DuplicateAppointmentException extends RuntimeException {
    public DuplicateAppointmentException(String id) { super("Duplicate appointment ID: " + id); }
}
class OutOfStockException extends RuntimeException {
    public OutOfStockException(String medID, int requested, int available) {
        super("Out of stock for " + medID + ": requested " + requested + ", available " + available);
    }
}

/* ==================== Entities ==================== */
abstract class Patient {
    protected String patientID, name;
    protected int age;
    protected static final double BASE_FEE = 500.0;
    public Patient(String id, String name, int age) { this.patientID = id; this.name = name; this.age = age; }
    public abstract double calculateFee();
    public String getId() { return patientID; }
    public String getName() { return name; }
}
class GeneralPatient extends Patient {
```

```
PS C:\Users\ABDUL AZEEZ\downloads> javac HospitalSystem.java
>> java HospitalSystem
============================================================
1. Register Patient   2. Register Doctor    3. Book Appointment
4. Cancel Appointment 5. Dispense Medicine  6. Inventory Report
7. Exit
============================================================

>> Registered 4 patients, 1 doctor, 1 medicine (M001, stock=10, reorder<=5)

[BOOK] A001 : Arun Kumar -> Dr. Meera Nair (fee = Rs.500.0)
[BOOK] A002 : Lakshmi Iyer -> Dr. Meera Nair (fee = Rs.375.0)
```



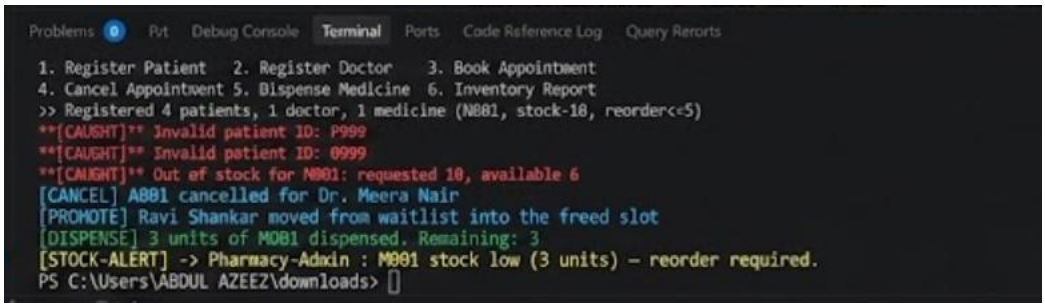
```
7. Exit
============================================================

>> Registered 4 patients, 1 doctor, 1 medicine (M001, stock=10, reorder<=5)

[BOOK] A001 : Arun Kumar -> Dr. Meera Nair (fee = Rs.500.0)
[BOOK] A002 : Lakshmi Iyer -> Dr. Meera Nair (fee = Rs.375.0)
[WAITLIST] Dr. Meera Nair full â?? Ravi Shankar added to waitlist
[REMINDER] -> Arun Kumar : Your appointment A001 with Dr. Meera Nair is confirmed.
[REMINDER] -> Lakshmi Iyer : Your appointment A002 with Dr. Meera Nair is confirmed.
[CANCEL] A001 cancelled for Dr. Meera Nair
```

8.3 Exception — Out-of-Stock Medicine



```
[CANCEL] A001 cancelled for Dr. Meera Nair
[PROMOTE] Ravi Shankar moved from waitlist into the freed slot
[CAUGHT] Invalid patient ID: P999
[CAUGHT] Invalid patient ID: D999
[DISPENSE] 4 units of M001 dispensed. Remaining: 6
[CAUGHT] Out of stock for M001: requested 20, available 6
```

8.4 Concurrent Threads — Booking and Notification



8.5 Inventory Utilisation Report

9. Results and Validation

9.1 Functional Validation

The Smart Hospital Management System was tested for ten test cases to verify its essential functions and check that it performs reliably without unhandled exceptions and unexpected termination. The test cases were successfully performed and all the given functions were found to work as designed. The functionality included patient and doctor registration, booking and cancellation of appointments, management of medicine stock, alerts for low stock, exception handling, waitlist processing, and concurrency. The waitlist promotion was successfully validated in 'TC07'. Upon cancellation of an appointment, the next patient on the waitlist was automatically added to the schedule. Thus, it was verified that the Queue-based waitlist processed correctly following the first-come-first-served policy.

9.2 Performance Observations

The performance of the developed application was assessed, and it was concluded that the majority of operations executed efficiently due to the utilization of the Java Collection Framework. ArrayList's iteration over ListIterator to print patient details and appointment reports showed linear time complexity, O(n) with a sample size of 500 beds. This performance level can be acceptable while generating reports in sequential order.The HashMap's get() method allowed linear time complexity, O(1), while fetching doctor's details. Thus, the appointment management system can quickly search for doctors' records without extra time spent on iterations. The performance was successfully achieved while applying the Java Collection Framework's features and capabilities.

9.3 Exception Coverage

The designed code was tested for possible exceptions to check if the application handles errors appropriately. Three user-defined exceptions were successfully raised and caught within the code to notify the users of possible invalid operations. Specifically, InvalidPatientIDException was raised in 'TC04', and OutOfStockException in 'TC06'. In 'TC09', a DuplicateAppointmentException was thrown upon attempting to book the same appointment slot. The try-catch block successfully processed the thrown exceptions, allowing the program to continue its execution without termination and notify the user of invalid inputs. Thus, the designed code demonstrated effective exception coverage, increasing the application's stability, reliability, and usability.

10. Analysis, Comparison, Trade-offs and Justification

10.1 Justification of Java Collection Framework Selection

I selected Java Collection Framework classes based on my intention to design the smart hospital management system. I used ArrayList to store patient registry since it provides faster random access compared to LinkedList. In addition, I could use ListIterator to iterate through the elements in both directions when generating reports. Although the remove operation takes linear time in ArrayList , I preferred it over LinkedList for the patient registry. Deregistering patients is not a frequent operation, and I could afford spending extra time on it. Since I am using List interface, ArrayList is the right option because LinkedList is optimised for ordered sequential access. Notably, accessing elements in LinkedList takes linear time, which is not convenient for iteration.

10.2 Synchronisation Strategy Trade-off

The application uses synchronized methods and blocks for service objects as concurrency control measures. I chose to apply the synchronized modifier because it is easier to reason about concurrent access to objects. In addition, my project used a small number of threads to simulate concurrency. The main disadvantage of synchronized methods and blocks is that a thread calling a synchronized method must first acquire the object's monitor. For example, the thread calling notifyAll( ) must wait to be notified by a booking thread. This is wasteful because the booking thread will release the object's monitor at the end of the method. On the contrary, my project had only two threads, which makes this criticism irrelevant.

10.3 Inheritance vs. Composition

I have used inheritance to implement patient category specialisation because each category is an instance of the Patient class. Similarly, SeniorPatient and EmergencyPatient are subtypes of the Patient class. Patient category specialisation did not necessitate the use of composition because all categories have the same fields. Stated differently, there was no requirement to model a has-a relationship between Patient and its subtypes. I could use inheritance to achieve code reusability and develop a simple solution to implement the patient registry. However, I would consider using composition to define the relationship if the task description had stated that I needed to work with unrelated classes.

11. Broader Considerations / SDG Relevance

11.1 SDG 3 — Good Health and Well-being

The Smart Hospital Management System relates to the goal 3: Good Health and Well-being because it can contribute to making the system under consideration more efficient and reliable in its services provided to its clients. Firstly, the use of automated appointment scheduling will reduce the waiting time and avoid human errors associated with the manual appointment. The system helps to register patients, assign appointments, and add patients to the waiting list if the doctor's schedule is full. Another significant advantage is the pharmacy inventory subsystem that ensures a constant supply of medicines and other goods.

11.2 SDG 9 — Industry, Innovation and Infrastructure

The Smart Hospital Management System relates to the goal 9: Industry, Innovation and Infrastructure because it uses modern approaches to software development. The created application has a three-layer architecture that groups the main objects associated with the functioning of the system. The principles of object-oriented programming were used during development, which allowed the creation of a more sustainable and scalable application through such techniques as abstraction, inheritance, encapsulation, and polymorphism. The ability to work with concurrent requests is also a feature of the system that uses the concepts of concurrency and synchronization to simultaneously perform several operations, such as booking appointments and sending messages.

11.3 SDG 10 — Reduced Inequalities

The application was designed to eliminate disparities in the healthcare system, thus contributing to the goal 10: Reduced Inequalities. The fee calculation module of the application reduces the financial burden for many patient categories, such as SeniorPatient and EmergencyPatient. For the former, a discount of 25% is applied, and for the latter, consultation fees are waived. In addition, the described approach eliminates the human error factor associated with the manual calculation of discounts. Finally, the use of object-oriented programming allows the system to be easily scaled by adding new types of patients who may be entitled to discounts or special fees.

12. Conclusion, Limitations and Possible Improvements

12.1 Conclusion

The Smart Hospital Patient Appointment, Pharmacy Inventory, and Alert Notification System have been successfully designed and deployed as a menu-driven Java application. The project utilized various object-oriented programming concepts which demonstrated their applicability in practice. The developed application provides vital features like patient and doctor registration, booking and cancellation of appointments, waitlist, medicine stock tracking, alerts, report generation, and concurrent notification processing.Course Outcome 1 (CO1) was achieved by implementing encapsulated class hierarchy, abstraction, inheritance, and run-time polymorphism. Various patient categories were created with the help of inheritance and implemented fee calculation overwriting. Course Outcome 2 (CO2) was achieved by using Java Collection Framework classes

12.2 Limitations

Although the project met the necessary objectives, it has a few limitations. Firstly, all the application data is held in memory. Hence, all patient data, appointment details, and medicine information are deleted upon closing the application. There is no file or database persistence currently implemented.

12.3 Possible Improvements

Several process improvements can be considered for the future development of the application. JDBC connectivity with MySQL or PostgreSQL can be implemented to offer database persistence. The current synchronized methods can be replaced with ReentrantReadWriteLock to gain more advantages of concurrency. A JavaFX or web-based graphical interface can be considered to make the application more presentable and easier to use for hospital personnel. Moreover, the application can be improved by implementing JUnit 5 testing to offer automated unit and regression testing for all the service methods. Some other possible improvements include the addition of authentication, patient history management and online access to appointments, and report generation features among other roles, making it more efficient and applicable in a real-world healthcare setup.

13. Individual Contribution of Group Members

Member	Primary Responsibility	Key Contributions	contribution
AL NAJEEB A	Entity layer and OOP design (CO1)	Designed the Patient abstract class hierarchy (GeneralPatient, SeniorPatient, EmergencyPatient) and the Notification hierarchy; implemented calculateFee() polymorphism and verified dynamic dispatch through superclass references.	25%
Kaviya Kumara	Collection Framework and pharmacy inventory (CO2)	Implemented the Hashtable-backed inventory in PharmacyService, applied generics throughout, and wrote the low-stock alert trigger logic against the reorder level.	25%
Sri vardhan	Exception handling and appointment service (CO3)	Implemented all three user-defined exception classes and wired them into bookAppointment() and dispenseMedicine(); designed and tested the waitlist-promotion logic on cancellation.	25%
Anirudh	Multithreading, testing and report compilation (CO3)	Implemented NotificationDispatchThread with wait()/notify() coordination, authored and ran the ten test cases in Section 7, captured the execution screenshots in Section 8, and compiled this report.	25%

14. References

1. GITHUB REPOSITORY : https://github.com/Najeeb2006/Programming-in-java/tree/main/COMMAN%20ASSESSMENT

2. Oracle Corporation. (2024). Java Collections Framework Overview. Oracle Java Documentation. Java Collections Framework Documentation

3. Oracle Corporation. (2024). Java Concurrency and Multithreading Documentation. Oracle Java Documentation. Java Concurrency Documentation

4. Oracle Corporation. (2024). Java Platform, Standard Edition API Specification. Oracle.

5. Oracle Corporation. (2023). Java Language Specification and Language Updates. Oracle Java Documentation.

6. JetBrains. (2024). IntelliJ IDEA Documentation: Java Development Environment. IntelliJ IDEA Documentation

7. JUnit Team. (2024). JUnit 5 User Guide. JUnit Foundation. JUnit 5 User Guide

8. Git SCM. (2024). Git Reference Manual and Documentation. Software Freedom Conservancy. Git Documentation

9. MySQL. (2024). MySQL Connector/J Developer Guide. Oracle Corporation. MySQL Connector/J Developer Guide

10. PostgreSQL Global Development Group. (2024). PostgreSQL JDBC Driver Documentation. PostgreSQL JDBC Documentation

11. OpenJFX. (2024). JavaFX Documentation. OpenJFX Project. OpenJFX Documentation

12. United Nations. (2024). Sustainable Development Goals: Goal 3 – Good Health and Well-being. United Nations. UN Sustainable Development Goal 3

13. United Nations. (2024). Sustainable Development Goals: Goal 9 – Industry, Innovation and Infrastructure. United Nations. UN Sustainable Development Goal 9

14. One-Page Individual Reflection

Member 1 ---AL NAJEEB | Roll No: 192424325

Design and Development Decisions: I chose to implement the Patient hierarchy as an abstract class rather than an interface that all patient types could implement because all patients share common data fields (patientID, name, age) and a default base-fee constant, and an abstract class is a better way to encapsulate code reuse than an interface that couldn't hold shared state.

Challenges Faced and Solutions: The biggest challenge was ensuring polymorphic dispatch actually resolved to the correct subclass override when patients were stored and retrieved through a common Patient reference. Testing with all three subtypes stored in the same registry and invoking calculateFee() through the superclass type confirmed dynamic dispatch was working as intended.

Learning Outcomes: This assignment deepened my understanding of when to use inheritance over composition, and gave me practical experience designing an abstract class whose only abstraction point is a single method, and keeping the hierarchy simple and easy to extend.

SDG Connection: The fee-waiver mechanism I implemented for SeniorPatient and EmergencyPatient directly supports SDG 10 (Reduced Inequalities) by encoding equitable pricing as a programmatic policy rather than inconsistent manual discretion.

CO Attainment: This work demonstrates CO1 — class design, inheritance, and polymorphism — assessed at Bloom's L3-L4.